

2026-05-18 - Device Tree Basics in Zephyr

Goal

Learn the fundamentals of Zephyr Device Tree and how hardware is described for your board.

What you will learn

- What a Device Tree is and why Zephyr uses it
- The structure of `.dts` and overlay files
- How Zephyr uses `device_tree.h` macros to access hardware
- Common node properties and bindings

Overview

Device Tree is a declarative hardware description language. In Zephyr, it describes peripherals, pin assignments, and board-specific configuration without hard-coding them in C.

Key concepts

- **Node:** a hardware block such as `i2c0`, `gpio0`, or `uart1`
- **Property:** settings like `reg`, `status`, `label`, `pinctrl-0`
- **Overlay:** custom modifications on top of the base board tree
- **Bindings:** JSON files that define allowed properties for a node type

Common macros

- `DT_NODELABEL(...)`
- `DT_ANY_INST_ON_BUS(...)`
- `DEVICE_DT_GET(...)`
- `DT_PROP(...)`

Why this matters

Device Tree allows portable hardware code, so the same application can run on different boards with only config changes.

Practice tasks

1. Open a board's `.dts` file and locate an I2C or GPIO node.
2. Create an `app.overlay` that enables a peripheral and changes its status to okay.
3. Write a tiny `main.c` that fetches the device with `DEVICE_DT_GET`.

Next topic

Tomorrow, we will study Kconfig and `prj.conf` configuration in Zephyr.